Tema 2 – Implantación de Arquitecturas Web



2.1 – Arquitecturas, componentes y entornos de una aplicación web

El despliegue de una aplicación web

Una aplicación puede funcionar correctamente en el equipo donde se ha desarrollado y fallar al trasladarla a otro sistema. El nuevo entorno podría utilizar versiones diferentes del software, carecer de una biblioteca necesaria o impedir el acceso a la base de datos. También pueden aparecer problemas relacionados con los permisos, las rutas, los puertos o la configuración.

El **despliegue** (***deployment*** en inglés) es el conjunto de operaciones necesarias para situar una aplicación en un entorno preparado para ejecutarla y ponerla a disposición de sus usuarios. No consiste únicamente en copiar archivos. Antes de comenzar hay que conocer:

- **La estructura de la aplicación**
- **Los componentes que intervienen**
- **Las dependencias necesarias**
- **El entorno de ejecución**
- **Los servicios externos utilizados**
- **Los requisitos de seguridad**
- **Las pruebas que permitirán comprobar el resultado**

Una aplicación estática puede necesitar solamente un servidor web. Una aplicación dinámica puede requerir un servidor de aplicaciones, una base de datos y otros servicios. La arquitectura determina qué componentes existen y cómo se relacionan.

En este apartado se utilizará **AulaDeploy** como hilo conductor. Su versión inicial es una aplicación estática formada por documentos HTML, una hoja CSS e imágenes. Esta primera versión permitirá observar el funcionamiento del despliegue sin que la programación oculte los elementos de la infraestructura.

Para explicar arquitecturas que incorporan lógica y persistencia también se considerará una evolución dinámica de AulaDeploy. Esta versión ampliada permitirá gestionar cursos y despliegues, autenticar usuarios, aplicar permisos y conservar información en una base de datos. Ambas versiones se utilizarán como ejemplos de arquitecturas diferentes y no deben confundirse: la primera solo necesita entregar archivos, mientras que la segunda requiere componentes adicionales.

Antes de instalar servidores es necesario comprender qué ocurre cuando una persona accede a una aplicación web.

Proceso de acceso a una aplicación web

El acceso comienza cuando una persona introduce una dirección en el navegador o selecciona un enlace. A partir de ese momento se desarrollan varias operaciones:



Si se solicita la página principal de AulaDeploy, el navegador puede recibir primero `index.html`. Al analizarlo, descubre referencias a `css/estilos.css`, `img/logotipo.svg` y otros recursos. Cada uno puede requerir una petición adicional.

La página que observa el usuario es, por tanto, el resultado de varias comunicaciones. Recibir correctamente el documento HTML no garantiza que todos los elementos asociados estén disponibles. Una ruta incorrecta puede provocar que falten estilos o imágenes.

Este recorrido permite identificar las piezas esenciales del sistema: cliente, servidor, recursos, direcciones, protocolos y respuestas.

Página web, sitio web y aplicación web

Una **página web** es un documento individual accesible mediante la Web. Generalmente está escrito en HTML y puede incorporar estilos, imágenes, vídeos, formularios y enlaces.

Un **sitio web** es un conjunto organizado de páginas y recursos relacionados que se publican bajo una identidad común. Sus páginas suelen compartir el mismo dominio, diseño y sistema de navegación.

Una **aplicación web** permite realizar tareas o participar en procesos mediante un navegador. Puede autenticar usuarios, procesar datos, almacenar cambios, consultar una base de datos o generar respuestas distintas según la petición recibida.

Las fronteras no siempre son absolutas. Un mismo producto puede combinar páginas informativas y funciones propias de una aplicación.

Desde el punto de vista del despliegue, lo importante es determinar qué elementos se entregan directamente y cuáles necesitan procesamiento, almacenamiento o servicios adicionales.

Concepto	Característica principal	Ejemplo
Página web	Documento individual	Página de contacto
Sitio web	Conjunto organizado de páginas	Sitio de un centro educativo
Aplicación web	Sistema que permite realizar operaciones	Aula virtual
Recurso web	Elemento identificable y accesible	Imagen, documento o curso

Arquitectura cliente-servidor

Las aplicaciones web se basan habitualmente en una **arquitectura cliente-servidor**. Este modelo distribuye el funcionamiento del sistema entre uno o varios componentes que solicitan servicios y otros componentes que los proporcionan. La comunicación se inicia cuando el cliente formula una petición y el servidor la recibe, la procesa y devuelve una respuesta. Esta separación permite que la interfaz utilizada por el usuario se encuentre en un dispositivo diferente del sistema que almacena los recursos o ejecuta la lógica de la aplicación.

El cliente

Inicia la comunicación. En una aplicación web suele ser el navegador, aunque también puede ser una aplicación móvil, una herramienta de línea de comandos u otro servicio.

El servidor

Permanece preparado para recibir peticiones y devolver respuestas. La palabra servidor puede referirse a dos elementos diferentes:

- **Máquina servidora:** equipo físico o virtual donde se ejecutan los servicios.
- **Software servidor:** programa que escucha peticiones y genera respuestas.

Cliente y servidor son funciones, no tipos permanentes de equipo. Un servidor web puede actuar como cliente cuando consulta una base de datos o solicita información a otro servicio.

Esta arquitectura permite centralizar los datos y la lógica, aplicar controles de acceso y actualizar el sistema sin modificar todos los dispositivos de los usuarios.

En AulaDeploy, el navegador actúa como cliente y solicita los archivos al servidor web. En una versión dinámica, el servidor podría consultar otros componentes antes de generar la respuesta.



Comunicación mediante peticiones y respuestas

HTTP utiliza un modelo de **petición y respuesta**. El cliente envía un mensaje que expresa qué recurso necesita o qué operación desea realizar. El servidor interpreta el mensaje y responde con el resultado.

Una petición puede:

- Solicitar una página.
- Enviar datos de un formulario.
- Crear un recurso.
- Actualizar información.
- Solicitar la eliminación de un elemento.

La respuesta puede:

- Entregar el recurso solicitado.
- Confirmar una operación.
- Redirigir al cliente.
- Solicitar autenticación.
- Comunicar un error.

HTTP se define como un protocolo sin estado. Cada petición debe contener la información necesaria para ser interpretada. El servidor no recuerda automáticamente todas las interacciones anteriores.

Las aplicaciones que necesitan conservar información entre peticiones incorporan mecanismos adicionales, como cookies, identificadores de sesión o tokens. Estos mecanismos permiten relacionar varias peticiones con un mismo usuario o proceso, aunque HTTP continúe tratando cada petición como un mensaje independiente.



Una respuesta no tiene que contener siempre HTML. También puede devolver una imagen, una hoja CSS, un archivo descargable, datos estructurados o una respuesta sin contenido.

Recursos y representaciones web

Un **recurso web** es cualquier elemento identificable con el que se puede interactuar mediante la Web. Puede ser un documento, una imagen, un usuario, un curso, una colección de datos o una operación proporcionada por una aplicación.

El recurso no debe confundirse con los datos concretos enviados al cliente. Un mismo recurso puede disponer de diferentes **representaciones**.

Un recurso no tiene que disponer de una única representación. Por ejemplo, la información correspondiente a un curso podría entregarse de distintas formas según el cliente que la solicite:

- Una página HTML para una persona.
- Datos JSON para otra aplicación.
- Un documento PDF descargable.
- Una versión adaptada a otro idioma.

En la versión estática inicial de AulaDeploy son recursos:

- La página de inicio.
- La página de cursos.
- La hoja de estilos.
- El logotipo y las demás imágenes.
- La documentación publicada.

En la evolución dinámica también pueden ser recursos:

- La ficha de un curso.
- El formulario de acceso.
- Un usuario identificado por la aplicación.
- Un registro de despliegue.
- El historial de operaciones.

Para poder solicitar un recurso es necesario identificarlo. La Web utiliza identificadores uniformes que permiten hacer referencia a elementos de naturaleza muy diferente mediante una sintaxis común.

Esta necesidad da lugar a los conceptos de URI, URL y URN.

Identificadores uniformes de recursos: URI

Una **URI**, siglas de *Uniform Resource Identifier*, es una secuencia de caracteres que identifica un recurso de manera uniforme.

Su sintaxis general comienza con un **esquema**, seguido de dos puntos y de una parte cuyo significado depende de ese esquema:

```
esquema:parte-específica
```

Ejemplos:

https

`https://auladeploy.example/cursos`

mailto

`mailto:soporte@auladeploy.example`

urn

`urn:isbn:9780000000000`

El esquema indica cómo debe interpretarse el resto del identificador. `https` se utiliza para recursos accesibles mediante HTTPS; `mailto` identifica una dirección de correo; `urn` permite construir nombres uniformes.

Una URI no tiene por qué corresponder a una página que pueda abrirse en el navegador. Su función general es identificar. Algunas URI proporcionan información para localizar el recurso, mientras que otras actúan principalmente como nombres.

En el uso cotidiano se emplea con mucha más frecuencia el término URL porque la mayoría de las direcciones web permiten localizar y solicitar un recurso.

URL y URN

Después de conocer el concepto general de URI, es necesario distinguir las dos categorías utilizadas tradicionalmente para identificar recursos: URL y URN. Ambas pertenecen al conjunto de las URI, pero expresan la identificación de manera diferente. Una URL indica dónde se encuentra un recurso o cómo puede accederse a él, mientras que una URN pretende identificarlo mediante un nombre estable dentro de un espacio de nombres. Esta diferencia permite comprender por qué ambos términos están relacionados, pero no son intercambiables.

URL — *Uniform Resource Locator*

Es una URI que permite localizar un recurso e indica el mecanismo utilizado para acceder a él.

```
https://auladeploy.example/cursos/despliegue.html
```

Esta URL contiene información sobre el protocolo, el sistema que ofrece el recurso y la ruta utilizada para solicitarlo.

URN — *Uniform Resource Name*

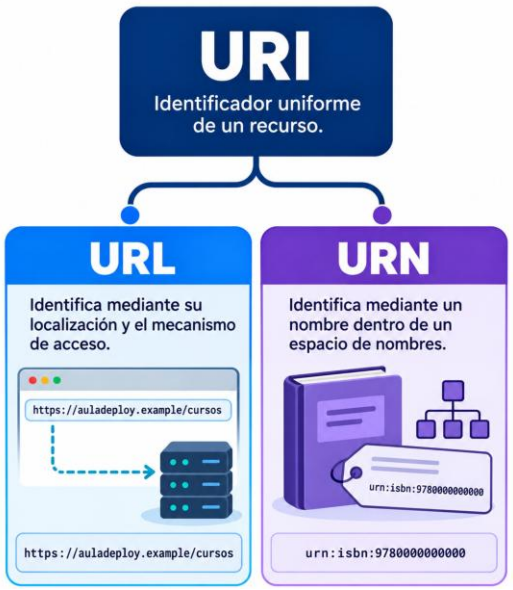
Identifica un recurso mediante un nombre dentro de un espacio de nombres.

```
urn:isbn:9780000000000
```

El ejemplo identifica una publicación mediante su ISBN, pero no indica directamente qué servidor debe consultarse para obtenerla.

Identificador	Función principal	Ejemplo
URI	Identificar un recurso	`mailto:soporte@example.com`
URL	Identificarlo mediante localización y acceso	`https://example.com/ayuda`
URN	Identificarlo mediante un nombre	`urn:isbn:...`

URL y URN no deben presentarse como sinónimos. En el despliegue web se trabaja principalmente con URL, pero URI es el concepto general que permite situarlas dentro de un sistema más amplio de identificación.



Estructura de una URL

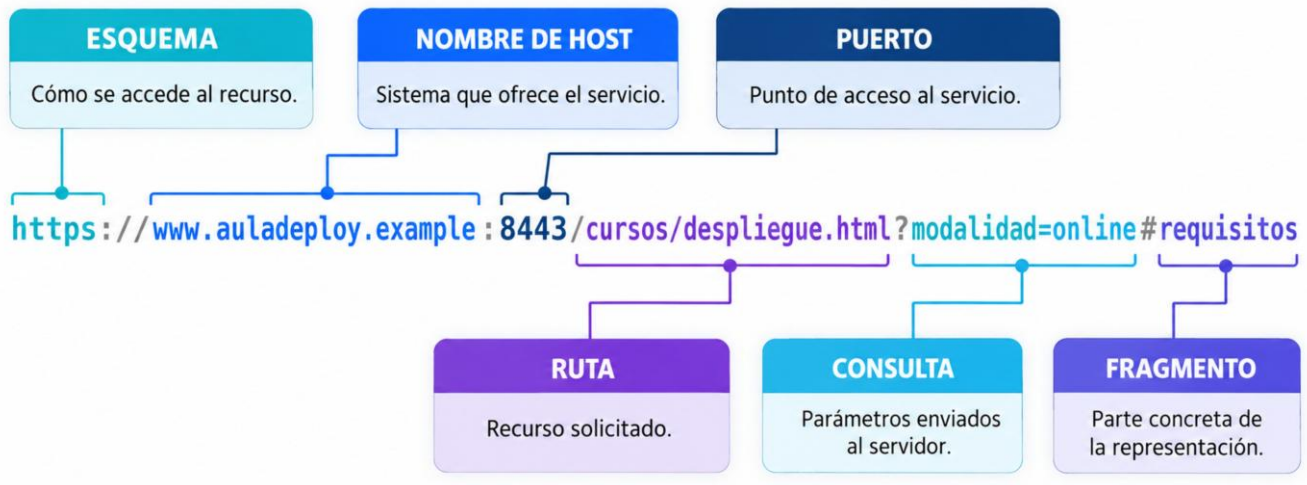
Una URL puede estar formada por varios componentes, cada uno de los cuales aporta información necesaria para localizar un recurso o solicitarlo correctamente. Algunos identifican el mecanismo de acceso y el sistema de destino; otros concretan el servicio, el recurso solicitado o determinados parámetros de la petición. No todas las URL contienen todos los componentes posibles, pero reconocerlos permite interpretar una dirección y diagnosticar numerosos errores de acceso y despliegue. Para analizarlos se utilizará la siguiente URL:

```
https://www.auladeploy.example:8443/cursos/despliegue.html?modalidad=online#requisitos
```

Componente	Valor	Función
Esquema	`https`	Indica cómo se accede al recurso
Nombre de host	`www.auladeploy.example`	Identifica el sistema de destino
Puerto	`8443`	Identifica el punto de acceso al servicio
Ruta	`/cursos/despliegue.html`	Señala el recurso solicitado
Consulta	`modalidad=online`	Proporciona parámetros al servidor
Fragmento	`requisitos`	Señala una parte de la representación

No todas las URL contienen todos estos componentes. Si el puerto no aparece, se utiliza normalmente el puerto predeterminado asociado al esquema. HTTP emplea habitualmente el 80 y HTTPS el 443.

El nombre de host debe asociarse a una dirección de red que permita alcanzar el sistema de destino. Esa resolución se realiza habitualmente mediante DNS. El nombre identifica el destino de una forma comprensible para las personas, mientras que la dirección permite encaminar la comunicación por la red.



Esquemas de URI

El **esquema** aparece al principio de una URI y termina con dos puntos. Determina cómo debe interpretarse el resto del identificador.

http

Acceso mediante HTTP.

https

Acceso mediante HTTP sobre una comunicación protegida.

mailto

Identificación de una dirección de correo.

ftp

Acceso mediante FTP.

file

Referencia a un archivo.

data

Inclusión de determinados datos en la propia URI.

urn

Identificación mediante un nombre uniforme.

Estas dos URL no son equivalentes:

```
http://auladeploy.example
https://auladeploy.example
```

Aunque comparten el mismo nombre de host, utilizan esquemas distintos y pueden dirigirse a servicios diferentes. HTTPS conserva la semántica de HTTP, pero protege la comunicación mediante TLS. Para establecer esa protección, el servidor debe disponer de una configuración criptográfica y de un certificado que permita acreditar su identidad.

El esquema no es una etiqueta decorativa. Condiciona el mecanismo de acceso, los puertos predeterminados y la forma de interpretar la URI.

Nombre de host y puerto

Una vez identificado el esquema de la URL, es necesario determinar a qué sistema y a qué servicio se dirigirá la comunicación. El nombre de host identifica el sistema de destino de una forma comprensible, mientras que el puerto señala el punto de acceso correspondiente al servicio que debe atender la petición. Ambos elementos trabajan de manera conjunta: dos direcciones con el mismo nombre de host pueden conducir a servicios diferentes si utilizan puertos distintos.

Nombre de host

Identifica el sistema al que se dirige la petición. Puede ser un nombre completo, como `www.auladeploy.example`, o una dirección IP.

Los nombres son más fáciles de recordar y permiten cambiar la dirección del sistema de destino sin obligar a los usuarios a conocer la nueva dirección IP. El servicio de resolución mantiene la relación necesaria entre el nombre utilizado y la dirección de red correspondiente.

Puerto

Permite distinguir varios servicios ejecutados en un mismo sistema. Una máquina puede ofrecer simultáneamente un servicio web, un servicio de transferencia y una base de datos.

Los puertos 80 y 443 son los habituales para HTTP y HTTPS. En desarrollo también son frecuentes puertos como 8080 o 8443.

Estas direcciones no identifican necesariamente el mismo servicio:

```
http://auladeploy.example
```

```
http://auladeploy.example:8080
```

```
https://auladeploy.example
```

Durante el desarrollo puede utilizarse una dirección como `localhost:8080`. En producción se suele proporcionar una URL más clara y evitar que el usuario tenga que introducir puertos no habituales.

El nombre de host dirige la comunicación hacia un sistema; el puerto conduce al servicio concreto que atiende dentro de ese sistema.

Ruta, cadena de consulta y fragmento

Después de identificar el sistema y el servicio de destino, la URL debe concretar qué recurso se solicita y, cuando sea necesario, aportar información adicional. Para ello pueden utilizarse la ruta, la cadena de consulta y el fragmento. Aunque aparecen dentro de una misma dirección, no cumplen la misma función: la ruta identifica el recurso, la consulta proporciona parámetros al servidor y el fragmento señala una parte concreta de la representación recibida.



Ruta

Identifica un recurso dentro del espacio organizado por el servidor:

```
/cursos/despliegue.html
```

No debe suponerse que cada segmento coincide con una carpeta física. El servidor o la aplicación pueden interpretar la ruta y generar una respuesta sin que exista un archivo con ese nombre.



Cadena de consulta

Comienza con `?` y permite incluir parámetros:

```
?modalidad=online&orden=fecha
```

Los parámetros suelen expresarse mediante parejas nombre-valor. El servidor puede emplearlos para buscar, filtrar, ordenar o seleccionar información.



Fragmento

Comienza con `#`:

```
#requisitos
```

Habitualmente señala una parte concreta de la representación. El navegador puede utilizarlo para desplazarse hasta un elemento de la página. El fragmento no se envía normalmente al servidor.

En la URL `https://auladeploy.example/cursos?modalidad=online#requisitos`, el servidor recibe la ruta `/cursos` y la consulta `modalidad=online`. Después de recibir la respuesta, el navegador utiliza `requisitos` para localizar la sección correspondiente.

URL absolutas y referencias relativas

Los documentos HTML pueden referirse a otros recursos mediante URL completas o mediante referencias que dependen de la ubicación del documento actual. Esta elección afecta directamente al despliegue, porque determina si los enlaces continuarán funcionando cuando la aplicación cambie de dominio, carpeta o entorno. Para interpretar correctamente una referencia relativa, el navegador debe combinarla con una URL base y construir la dirección final del recurso.

URL absoluta

Contiene la información necesaria para identificar el recurso sin depender de otra dirección:

```
https://auladeploy.example/img/logotipo.svg
```

Referencia relativa

Se interpreta tomando como base la ubicación del documento actual:

```
img/logotipo.svg
```

Si el documento actual se encuentra en `https://auladeploy.example/cursos/index.html`, la referencia se resolverá como:

```
https://auladeploy.example/cursos/img/logotipo.svg
```

Una referencia que comienza por `/` parte de la raíz del sitio. Su resultado será:

```
https://auladeploy.example/img/logotipo.svg
```

Consideraciones de despliegue

Una aplicación preparada para su despliegue debe evitar rutas que solo funcionen en el equipo del desarrollador, como ubicaciones locales del sistema de archivos o nombres internos que no existen en producción.

Las referencias relativas facilitan el traslado de una aplicación entre entornos, pero pueden producir errores si se desconoce la dirección base. Las URL absolutas aportan claridad, aunque pueden quedar vinculadas a un dominio concreto.

Protocolo HTTP

La comunicación entre un cliente y un servidor web requiere un conjunto de reglas compartidas que determine cómo deben construirse, transmitirse e interpretarse los mensajes. HTTP, siglas de *Hypertext Transfer Protocol*, es el protocolo de la capa de aplicación que establece esas reglas. Define cómo expresa el cliente la operación que desea realizar, cómo se identifica el recurso afectado y cómo comunica el servidor el resultado de la petición. También permite describir las características del contenido intercambiado mediante información adicional incluida en los mensajes.

HTTP define:

- Cómo expresa el cliente la operación solicitada.
- Cómo identifica el recurso de destino.
- Cómo comunica el servidor el resultado.
- Cómo se describen las representaciones enviadas.
- Qué información adicional pueden incluir los mensajes.

Las versiones actuales mantienen la misma semántica general:

HTTP/1.1

Emplea una representación textual reconocible.

HTTP/2

Permite multiplexar varios intercambios sobre una conexión.

HTTP/3

Utiliza QUIC como transporte.

Para comprender el despliegue no es necesario estudiar el funcionamiento interno completo de cada versión. Lo fundamental es reconocer métodos, cabeceras, códigos de estado y contenido.

HTTP es extensible. Puede incorporar nuevas cabeceras, métodos o tipos de contenido sin modificar su modelo esencial de petición y respuesta.

Petición HTTP

Cada interacción HTTP comienza con una petición enviada por el cliente. Este mensaje debe indicar qué operación desea realizar, sobre qué recurso quiere actuar y qué información adicional necesita conocer el servidor para interpretar la solicitud. Aunque una petición puede representarse de distintas formas según la versión de HTTP utilizada, conceptualmente se organiza en tres partes: una línea inicial, un conjunto de cabeceras y, cuando la operación lo requiere, un cuerpo.

Ejemplo simplificado

```
GET /cursos/despliegue.html HTTP/1.1
Host: auladeploy.example
Accept: text/html
User-Agent: navegador-ejemplo
```

Explicación de cada elemento

- `GET` es el método y expresa que el cliente desea obtener una representación.
- `/cursos/despliegue.html` es el objetivo de la petición.
- `HTTP/1.1` identifica la versión empleada para representar el mensaje.
- `Host` comunica el nombre al que se dirige la petición.
- `Accept` expresa los tipos de contenido que el cliente admite.

La cabecera `Host` es especialmente importante cuando un servidor atiende varios sitios. Algunas peticiones incluyen un cuerpo: es habitual cuando se envían los datos de un formulario o una representación que debe crearse o modificarse. Una petición GET ordinaria utiliza la URL y las cabeceras para solicitar el recurso y no necesita un cuerpo.

Respuesta HTTP

Después de recibir e interpretar una petición, el servidor genera una **respuesta HTTP**. Este mensaje informa al cliente del resultado de la operación y puede incluir la representación del recurso solicitado, instrucciones para continuar el acceso o información sobre un error. Conceptualmente, la respuesta se organiza en una línea de estado, un conjunto de cabeceras y un cuerpo opcional. El cliente utilizará estos elementos para decidir cómo debe tratar el resultado recibido.

Ejemplo simplificado

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=UTF-8
Content-Length: 4821

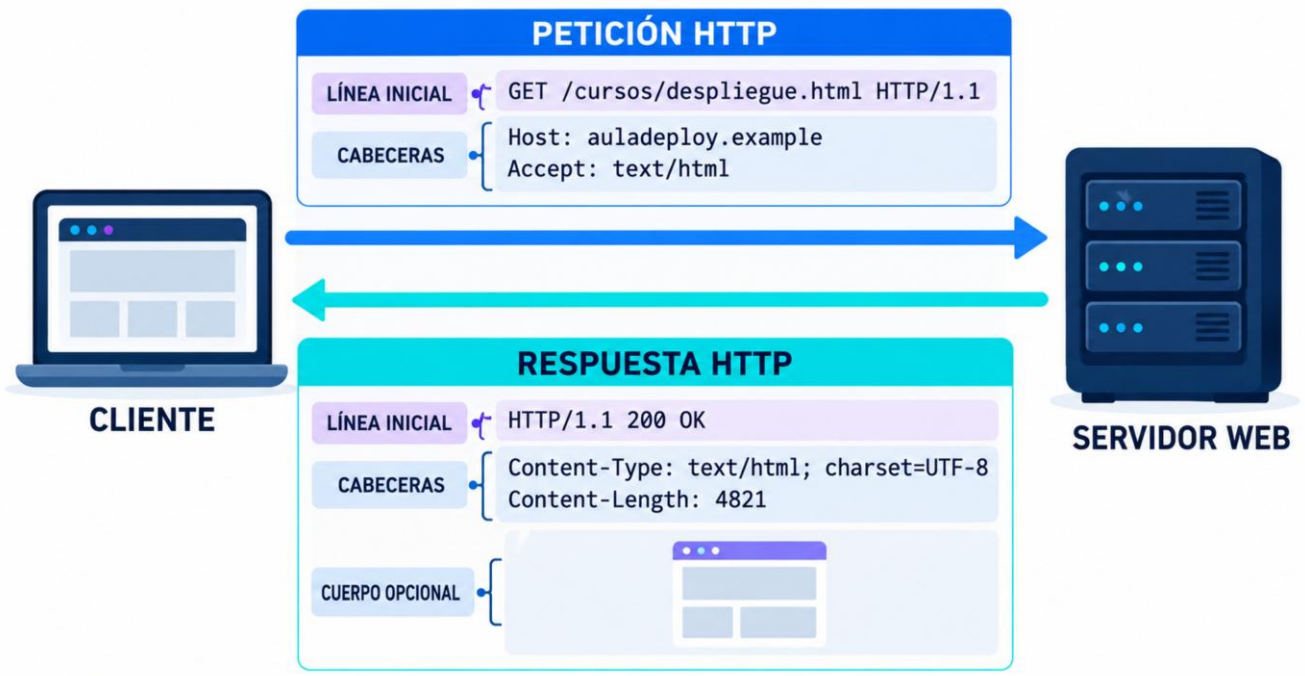
<!doctype html>
<html lang="es">
...
</html>
```

Explicación de cada elemento

- El código `200` indica que la petición se ha atendido correctamente. La expresión `OK` facilita la lectura humana.
- `Content-Type` informa de que la representación es un documento HTML codificado en UTF-8.
- `Content-Length` comunica la longitud del contenido cuando el servidor la conoce y decide indicarla.
- Tras una línea en blanco comienza el cuerpo del mensaje.

No todas las respuestas contienen cuerpo. Una redirección puede limitarse a indicar otra ubicación. Una operación puede completarse correctamente sin devolver contenido. También puede enviarse una representación explicativa junto a un error.

El navegador interpreta el código, las cabeceras y el cuerpo para decidir cómo debe tratar la respuesta.



Métodos HTTP

Las peticiones HTTP no se diferencian únicamente por el recurso al que se dirigen. También deben indicar la intención de la operación que el cliente desea realizar. Esa intención se expresa mediante el **método HTTP** incluido en la línea inicial de la petición. El método permite distinguir, por ejemplo, entre obtener una representación, enviar información para que sea procesada, actualizar un recurso o solicitar su eliminación. Su significado orienta al servidor, aunque la aplicación debe decidir finalmente si admite la operación y si el usuario dispone de autorización.

Método	Finalidad habitual
`GET`	Obtener una representación
`HEAD`	Obtener únicamente las cabeceras
`POST`	Enviar información para que sea procesada
`PUT`	Crear o sustituir una representación
`PATCH`	Aplicar una modificación parcial
`DELETE`	Solicitar la eliminación de un recurso
`OPTIONS`	Consultar las opciones disponibles

El resultado depende de la aplicación. Enviar `DELETE` no obliga por sí solo a eliminar un recurso. El servidor debe admitir el método, identificar al usuario y comprobar sus permisos.

Métodos seguros

Un método es seguro cuando su significado indica una operación de consulta y el cliente no solicita una modificación del estado del recurso. Entre los métodos incluidos en la tabla, GET, HEAD y OPTIONS son seguros. El servidor puede realizar acciones internas, como registrar la petición, pero esas acciones no constituyen la finalidad solicitada por el cliente.

Métodos idempotentes

Un método es idempotente cuando varias peticiones idénticas producen sobre el servidor el mismo efecto previsto que una sola petición. Entre los métodos incluidos en la tabla son idempotentes GET, HEAD, OPTIONS, PUT y DELETE. Las respuestas pueden ser diferentes: el primer DELETE puede eliminar un recurso y el segundo informar de que ya no existe, pero el estado final continúa siendo el mismo. POST y PATCH no tienen por qué ser idempotentes.

Estas propiedades ayudan a decidir si una petición puede repetirse después de un fallo de comunicación. También influyen en el funcionamiento de intermediarios, cachés y herramientas automáticas.

Códigos de estado HTTP

Después de procesar una petición, el servidor debe comunicar al cliente si la operación se ha realizado correctamente, si necesita alguna actuación adicional o si se ha producido un problema. Esta información se expresa mediante el código de estado incluido en la primera línea de la respuesta HTTP.

El código permite que el navegador, otra aplicación cliente o una herramienta de diagnóstico interpreten el resultado sin depender únicamente del contenido visible de la respuesta. Su primera cifra identifica la categoría general del resultado, por lo que los códigos se organizan en cinco grupos:

Grupo	Significado	Ejemplos
1xx	Información provisional	100
2xx	Operación completada	200, 201, 204
3xx	Redirección	301, 302, 304
4xx	Problema relacionado con la petición	400, 401, 403, 404
5xx	Error o incapacidad del servidor para atender una petición	500, 502, 503

Códigos frecuentes:

- **200 OK:** petición atendida correctamente.
- **201 Created:** recurso creado.
- **301 Moved Permanently:** nueva ubicación permanente.
- **400 Bad Request:** petición incorrecta.
- **401 Unauthorized:** la petición carece de credenciales de autenticación válidas.
- **403 Forbidden:** el servidor ha comprendido la petición, pero rechaza autorizarla.
- **404 Not Found:** recurso no encontrado.
- **500 Internal Server Error:** error interno.
- **503 Service Unavailable:** servicio temporalmente no disponible.

El código permite clasificar el resultado, pero no siempre explica la causa. Para diagnosticar un problema es necesario relacionarlo con la petición, la configuración y los registros del servidor.

Diferencia entre 401 y 403

Un código 401 indica que falta una autenticación válida o que las credenciales presentadas no son aceptables. Un código 403 indica que el servidor conoce la petición, pero no permite realizarla con los permisos disponibles. Por ejemplo, una persona sin sesión iniciada podría recibir un 401, mientras que un usuario autenticado sin permisos de administración podría recibir un 403.

Cabeceras HTTP

El método y la dirección del recurso no siempre proporcionan información suficiente para interpretar una petición o una respuesta. Los mensajes HTTP incorporan **cabeceras** que transportan metadatos sobre la comunicación, el cliente, el servidor o el contenido enviado. Mediante estas cabeceras pueden indicarse los formatos aceptados, el tipo y la longitud del contenido, las condiciones de almacenamiento temporal, una nueva ubicación o determinadas credenciales. Su significado puede modificar la manera en que se procesa el mensaje.

Cabeceras habituales:

- `Host:` nombre al que se dirige la petición.
- `Accept:` tipos de contenido que el cliente admite.
- `Content-Type:` tipo del contenido enviado.
- `Content-Length:` longitud del cuerpo.
- `Location:` nueva ubicación de un recurso.
- `Cache-Control:` instrucciones de almacenamiento temporal.
- `Authorization:` credenciales o información de autorización.
- `User-Agent:` información declarada por el cliente.
- `Server:` información que el servidor decide proporcionar.

Consideraciones importantes

Las cabeceras no son comentarios. Pueden controlar la caché, indicar formatos, proporcionar credenciales o modificar el tratamiento del mensaje.

Tampoco debe confiarse automáticamente en todas las cabeceras enviadas por el cliente. Algunos valores pueden manipularse. Si afectan a una decisión de seguridad, la aplicación debe validarlos.

Durante el despliegue, la revisión de cabeceras permite detectar errores de tipo de contenido, redirecciones, caché, autenticación o exposición innecesaria de información.

Carga del documento HTML y sus recursos asociados

El navegador no recibe normalmente todos los elementos de una página web mediante una única petición. Primero solicita el documento HTML y comienza a analizar su contenido. Cuando encuentra referencias a hojas de estilo, imágenes, fuentes u otros recursos, genera nuevas peticiones para obtener cada uno de ellos.

Este comportamiento explica por qué una página puede mostrar el texto principal y, sin embargo, aparecer sin estilos o con imágenes ausentes. El documento HTML ha llegado correctamente, pero alguna de las peticiones posteriores ha fallado o utiliza una ruta incorrecta. Supóngase que `index.html` contiene las siguientes referencias:

```
<link rel="stylesheet" href="css/estilos.css">
<img src="img/logotipo.svg" alt="AulaDeploy">
```



Si la hoja de estilos no se encuentra, el documento HTML puede mostrarse sin diseño. Si falla la imagen, el resto de la página puede continuar disponible.

Por ello, una comprobación de despliegue no debe limitarse a abrir la portada. También hay que revisar los recursos vinculados, las rutas, los códigos de estado, los tipos de contenido y las herramientas de desarrollo del navegador. Una sola página puede implicar numerosas peticiones a uno o varios servidores.

Contenido web estático

Un recurso web es **estático** cuando el servidor puede entregarlo directamente a partir de un archivo almacenado, sin ejecutar lógica para generar una respuesta específica para cada solicitud. Ante peticiones equivalentes, el servidor proporciona normalmente el mismo contenido, aunque las cabeceras o las condiciones de almacenamiento temporal puedan variar.

La publicación de contenido estático suele ser sencilla porque no requiere un entorno de ejecución para procesar la petición ni una base de datos para construir la respuesta. Sin embargo, el despliegue debe conservar la estructura de archivos, las rutas y los permisos necesarios. Un documento HTML puede recibirse correctamente y aparecer incompleto si no se han publicado sus hojas de estilo, imágenes o fuentes asociadas.

Son recursos estáticos habituales:

- Documentos HTML.
- Hojas de estilo CSS.
- Imágenes.
- Fuentes.
- Documentos descargables.
- Archivos de audio o vídeo.

La versión inicial de AulaDeploy es estática. El servidor recibe la petición, localiza el archivo y devuelve su contenido. No necesita ejecutar lógica de negocio ni consultar una base de datos.

Estático no significa inmutable. Los archivos pueden modificarse y publicarse de nuevo. El término indica que la respuesta no se construye dinámicamente para cada petición.

Ventajas habituales:

Despliegue sencillo

Bajo consumo de recursos

Pocas dependencias

Facilidad para utilizar caché

Menor superficie de ataque

Su limitación aparece cuando el sistema debe procesar operaciones, adaptar respuestas a cada usuario o conservar información. En ese caso se necesita algún componente dinámico.

Contenido web dinámico

El servidor no siempre dispone de un archivo terminado que pueda entregar directamente. En una aplicación dinámica, la respuesta se construye en el momento de atender la petición mediante la ejecución de lógica. El resultado puede depender de la dirección solicitada, los datos enviados, la identidad del usuario, sus permisos o la información almacenada en una base de datos. Por esta razón, dos peticiones dirigidas a una misma ruta no tienen que producir necesariamente la misma respuesta.

Al solicitar la ficha de un curso, la aplicación podría:

01
Leer el identificador de la URL

02
Comprobar la identidad del usuario

03
Verificar sus permisos

04
Consultar una base de datos

05
Aplicar reglas de negocio

06
Construir la respuesta

07
Devolver HTML o datos estructurados

Dos usuarios pueden solicitar la misma ruta y recibir respuestas diferentes según sus permisos, preferencias o información almacenada.

El contenido dinámico permite desarrollar sistemas de matrícula, comercio electrónico, gestión académica y administración interna. A cambio, requiere más componentes y una configuración más cuidadosa.

Una aplicación dinámica también contiene recursos estáticos. La ficha puede generarse en el servidor mientras sus estilos, imágenes y fuentes se entregan directamente.

Contenido estático y contenido dinámico no son modelos excluyentes. Lo habitual es que ambos convivan dentro de la misma aplicación.



Procesamiento en el cliente y en el servidor

El **lado cliente** comprende los elementos procesados en el dispositivo del usuario. El navegador analiza el HTML, aplica CSS, solicita recursos y construye la interfaz.

El **lado servidor** comprende la lógica y los datos procesados dentro de la infraestructura que presta el servicio.

Lado cliente	Lado servidor
Presenta la interfaz	Aplica reglas de negocio
Interpreta HTML y CSS	Valida operaciones
Solicita recursos	Consulta bases de datos
Facilita interacción	Controla permisos
Muestra resultados	Genera respuestas

El navegador no debería conectarse directamente a la base de datos. Las operaciones sensibles deben pasar por componentes controlados del servidor.

En AulaDeploy, HTML y CSS se procesan en el navegador. Si se incorporase una matrícula conectada a una base de datos, el servidor tendría que validar y almacenar la operación.

Una validación realizada en el cliente puede mejorar la experiencia de uso, pero no sustituye a la validación que protege el servidor.

Servidor web

Un **servidor web** es un programa que recibe peticiones HTTP y devuelve respuestas. Puede entregar archivos almacenados o trasladar la petición a otro componente capaz de generar la respuesta.

Funciones habituales:

- Escuchar en determinados puertos.
- Atender uno o varios sitios.
- Localizar recursos.
- Determinar tipos de contenido.
- Aplicar reglas de acceso.
- Registrar peticiones y errores.
- Redirigir solicitudes.
- Proteger comunicaciones mediante HTTPS.
- Reenviar peticiones a aplicaciones internas.

Apache HTTP Server y Nginx son ejemplos de servidores web.

En un sitio estático, el servidor web puede ser el componente principal. En una aplicación dinámica puede actuar como punto de entrada y colaborar con un servidor de aplicaciones.

El servidor web es software. Puede ejecutarse en un servidor físico, una máquina virtual, una infraestructura en la nube o un contenedor.

La elección de un servidor web depende del tipo de contenido que debe publicar, los protocolos admitidos, los módulos disponibles, el sistema operativo, los requisitos de seguridad y la forma en que deba comunicarse con otros componentes.

Servidor de aplicaciones

Cuando una aplicación debe ejecutar lógica en el servidor, necesita un entorno capaz de cargar su código, proporcionarle los servicios requeridos y controlar su funcionamiento. Esa responsabilidad corresponde al **servidor de aplicaciones** o al entorno de ejecución equivalente de la plataforma utilizada. No se limita a entregar archivos: participa en la ejecución de las operaciones dinámicas y puede gestionar el ciclo de vida de la aplicación, sus conexiones, sesiones, recursos compartidos y mecanismos de integración.

Según la plataforma, puede encargarse de:

- Cargar y ejecutar aplicaciones.
- Administrar su ciclo de vida.
- Proporcionar bibliotecas compartidas.
- Gestionar conexiones.
- Aplicar mecanismos de autenticación.
- Mantener sesiones.
- Facilitar acceso a recursos.
- Registrar errores y eventos.
- Integrarse con otros servidores.

Apache Tomcat es un contenedor web para aplicaciones Java basadas en servlets y tecnologías relacionadas. Aunque no incorpora todos los servicios de un servidor Jakarta EE completo, desempeña el papel de entorno de ejecución para numerosas aplicaciones web Java.

La plataforma elegida también condiciona el despliegue. En este punto no es necesario repetir la diferencia entre software libre y propietario, ya conocida, sino analizar sus consecuencias prácticas: condiciones de licencia, sistemas operativos compatibles, disponibilidad de actualizaciones, documentación, soporte técnico, herramientas de administración y facilidad para trasladar la aplicación a otra plataforma.

El servidor de aplicaciones puede recibir peticiones directamente o situarse detrás de un servidor web. En este segundo caso, el servidor web atiende la comunicación exterior y reenvía las operaciones dinámicas.

Para seleccionar un servidor de aplicaciones es necesario comprobar que admite la plataforma y la versión utilizadas por la aplicación, que reconoce su formato de empaquetado y que proporciona los servicios requeridos. También deben valorarse su configuración, actualización, documentación, herramientas de administración y procedimiento de recuperación.

Base de datos

Muchas aplicaciones necesitan conservar información después de que finalice una petición o se reinicie el servidor. Para proporcionar esta persistencia se utiliza habitualmente una base de datos, que permite almacenar, consultar y modificar información de forma organizada. La aplicación actúa como intermediaria entre el usuario y los datos: recibe la operación solicitada, comprueba su validez y sus permisos, aplica las reglas correspondientes y solo entonces realiza la consulta o modificación necesaria.

Puede almacenar:

- Usuarios.
- Cursos.
- Matrículas.
- Roles y permisos.
- Pedidos.
- Mensajes.
- Configuraciones.
- Historial de operaciones.

La base de datos puede encontrarse:

- En la misma máquina que la aplicación.
- En otra máquina de la red.
- En una máquina virtual independiente.
- En un contenedor.
- En un servicio gestionado en la nube.

El navegador no debe acceder directamente a la base de datos. La petición se dirige a la aplicación, que valida la operación, aplica las reglas necesarias y realiza la consulta. Esta separación protege la estructura interna de los datos y permite controlar los permisos.

El despliegue debe conocer el tipo de base de datos, la dirección, el puerto, las credenciales, el esquema y el procedimiento de inicialización. Las credenciales no deben almacenarse en el código publicado.

Servicios auxiliares y dependencias externas

Una aplicación puede depender de servicios que no forman parte de su núcleo principal, pero que proporcionan funciones necesarias. Estas dependencias deben identificarse antes del despliegue porque cada una incorpora una dirección, un protocolo, una configuración y una posible causa de fallo.

Cada servicio amplía las capacidades del sistema, pero introduce nuevas condiciones de funcionamiento. Antes del despliegue conviene registrar:

Servicio auxiliar	Función que proporciona	Consecuencia de su indisponibilidad
Almacenamiento de archivos	Conserva documentos, imágenes o archivos subidos por los usuarios	La aplicación puede funcionar, pero no podrá recuperar o guardar esos archivos
Caché	Mantiene temporalmente datos o respuestas para evitar operaciones repetidas	Aumentará el tiempo de respuesta o la carga sobre otros componentes
Cola de mensajes	Conserva tareas que deben procesarse de forma asíncrona	Las tareas pueden retrasarse o quedar pendientes
Servicio de identidad	Autentica usuarios o proporciona información sobre sus permisos	Puede impedir el inicio de sesión o el acceso a funciones protegidas
API externa	Proporciona datos u operaciones pertenecientes a otro sistema	Las funciones que dependan de ella dejarán de estar disponibles
Buscador	Indexa información y permite realizar búsquedas especializadas	Las búsquedas pueden fallar aunque el resto de la aplicación continúe funcionando
Registro y monitorización	Recoge información sobre el funcionamiento y detecta incidencias	La aplicación puede seguir activa, pero será más difícil detectar y diagnosticar fallos

No todas las dependencias tienen la misma importancia. Algunas son imprescindibles para que la aplicación arranque; otras afectan únicamente a una función secundaria. El inventario de implantación debe indicar para cada servicio su ubicación, protocolo, mecanismo de autenticación, entorno correspondiente y comportamiento previsto cuando no esté disponible.

En AulaDeploy, la base de datos sería una dependencia esencial de la edición dinámica. Un servicio de correo utilizado para enviar avisos podría considerarse secundario si su fallo no impide consultar o gestionar los despliegues.

Componentes de una arquitectura web

Describir la arquitectura de una aplicación web no consiste únicamente en enumerar los programas utilizados. Es necesario identificar los componentes que forman el sistema, asignar a cada uno una responsabilidad y explicar las relaciones que mantienen. Esta descripción permite comprender dónde se presenta la interfaz, dónde se ejecuta la lógica, dónde se almacenan los datos y qué comunicaciones deben funcionar para que la aplicación preste el servicio esperado.

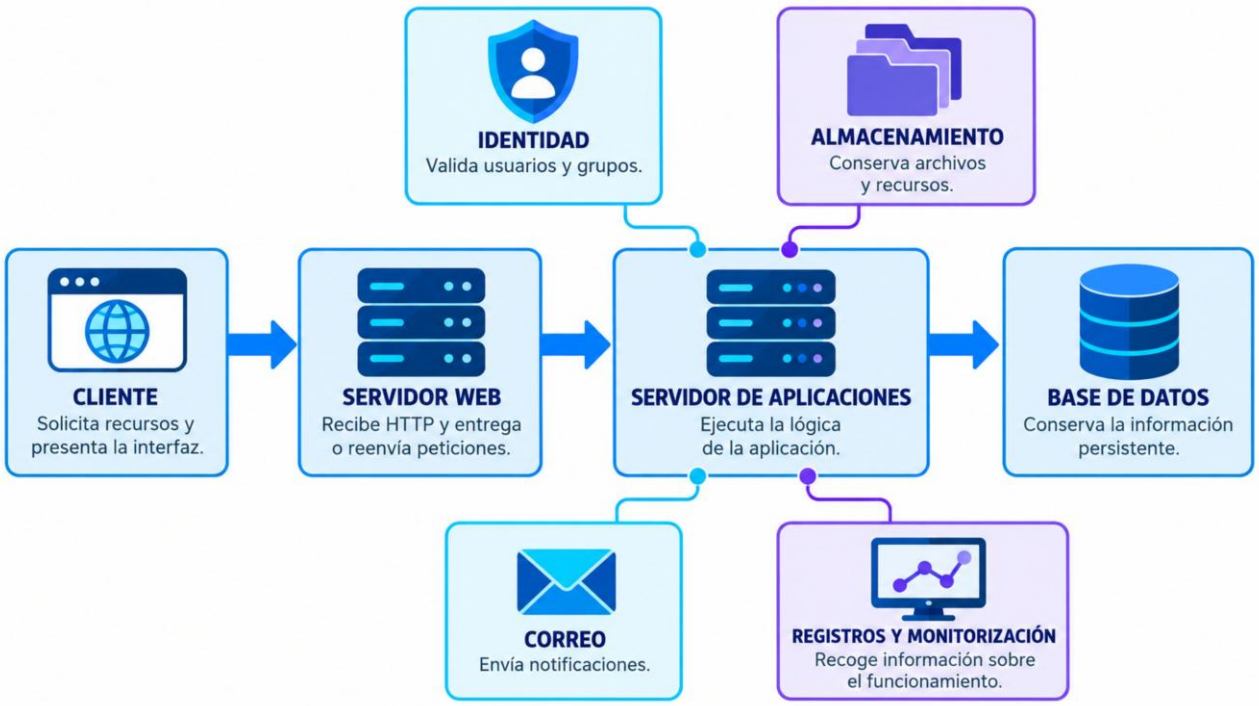
Los componentes más habituales son:

- Cliente.
- Servidor web.
- Servidor de aplicaciones.
- Base de datos.
- Servicios auxiliares.
- Sistemas externos.
- Elementos de almacenamiento.
- Herramientas de supervisión.

Una descripción arquitectónica debe permitir responder:

- ¿Dónde se presenta la interfaz?
- ¿Dónde se ejecuta la lógica?
- ¿Dónde se almacenan los datos?
- ¿Qué componente recibe las peticiones?
- ¿Cómo se comunican los componentes?
- ¿Qué dependencias externas existen?
- ¿Qué elementos pueden desplegarse por separado?
- ¿Dónde se aplican los controles de seguridad?

La arquitectura lógica representa funciones y capas. La arquitectura física representa máquinas, redes y ubicaciones. La arquitectura de despliegue relaciona los componentes software con la infraestructura donde se ejecutan. Un diagrama debe representar la información necesaria para comprender el sistema, evitando detalles que oculten las decisiones principales.



Arquitectura en dos capas

Una arquitectura en dos capas divide el sistema en dos bloques principales: cliente y servidor. En un sitio web estático, el navegador solicita recursos y el servidor web los entrega. El servidor puede concentrar diferentes responsabilidades, como lógica y acceso a datos.

En este modelo, «dos capas» se refiere a la separación entre el cliente y el servidor. No significa que existan obligatoriamente dos equipos físicos. Durante una práctica, el navegador y el servidor pueden ejecutarse en el mismo ordenador y seguir cumpliendo funciones distintas.



Ventajas:

- Instalación sencilla.
- Pocos componentes.
- Administración directa.
- Diagnóstico accesible.
- Coste inicial reducido.

Limitaciones:

- El servidor puede acumular demasiadas responsabilidades.
- Las funciones no siempre pueden escalarse de forma independiente.
- El crecimiento puede dificultar el mantenimiento.
- Un único servidor puede convertirse en un punto de fallo.

La versión estática inicial de AulaDeploy responde a este modelo: el navegador constituye la capa cliente y el servidor web se encarga de entregar los documentos HTML, las hojas de estilo y las imágenes solicitadas.

Arquitectura en tres capas

Cuando la presentación, el procesamiento de las operaciones y la conservación de los datos tienen necesidades diferentes, resulta conveniente separar estas responsabilidades. La arquitectura en tres capas organiza el sistema en una capa de presentación, una capa de lógica de aplicación y una capa de datos. Cada una realiza una función concreta y se comunica con las demás mediante relaciones definidas. Esta división es lógica: no obliga a utilizar tres máquinas ni tres servidores físicos distintos.



En una versión dinámica de AulaDeploy, el navegador compondría la capa de presentación, la aplicación ejecutada en Tomcat procesaría la lógica y H2 conservaría los datos. La petición atravesaría estas responsabilidades, aunque todos los componentes podrían ejecutarse inicialmente en una sola máquina.

Ventajas:

- Separación de responsabilidades.
- Mayor facilidad de mantenimiento.
- Menor exposición de los datos.
- Posibilidad de modificar una capa con menor impacto.
- Asignación diferenciada de recursos.

Inconvenientes:

- Más componentes que configurar.
- Mayor número de comunicaciones.
- Diagnóstico más complejo.
- Posibles problemas de compatibilidad entre capas.

Las capas representan responsabilidades lógicas. Pueden ejecutarse dentro de una sola aplicación o distribuirse entre varios procesos y máquinas. Cuando se habla de niveles físicos de despliegue se describe dónde se ejecuta cada componente. Por tanto, una aplicación organizada en tres capas no tiene que utilizar necesariamente tres servidores.

Arquitectura multicapa

Una arquitectura multicapa amplía la separación de responsabilidades cuando la aplicación necesita componentes especializados. Puede incorporar un servidor web de entrada, uno o varios servidores de aplicaciones, sistemas de persistencia y servicios auxiliares. Cada componente desempeña una función concreta y puede disponer de un ciclo de despliegue, una configuración y unos recursos propios.



El servidor web puede encargarse de los recursos estáticos y recibir las conexiones externas. El servidor de aplicaciones procesa las operaciones dinámicas. La base de datos conserva la información. También pueden incorporarse caché, almacenamiento, servicio de identidad, cola de mensajes, buscador y sistema de monitorización.

Distribuir las responsabilidades entre varios componentes puede mejorar el aislamiento, la administración y el escalado selectivo. Por ejemplo, sería posible ampliar únicamente los servidores de aplicaciones si la lógica constituye el principal límite del sistema, sin tener que reproducir todos los demás componentes.

Esta distribución también aumenta la complejidad. Cada nuevo componente necesita configuración, comunicaciones de red, supervisión y control de compatibilidad. Además, puede convertirse en un nuevo punto de fallo. Una arquitectura multicapa no consiste simplemente en ejecutar la misma aplicación en más máquinas: cada componente debe asumir una responsabilidad identificable y su incorporación debe responder a un requisito concreto.

Arquitectura monolítica

En una arquitectura monolítica, la mayor parte de la funcionalidad se construye y despliega como una única aplicación. Un monolito puede estar bien organizado internamente mediante módulos. Su característica principal es que esos módulos se publican juntos.

Ventajas:

- Despliegue inicial sencillo.
- Menor número de servicios.
- Comunicación interna directa.
- Diagnóstico más accesible en sistemas pequeños.
- Menor complejidad operativa.

Limitaciones:

- Una modificación puede exigir desplegar toda la aplicación.
- Los módulos no se escalan de forma independiente.
- Un fallo puede afectar a numerosas funciones.
- El crecimiento puede dificultar la coordinación.

Una distinción importante:

La arquitectura monolítica no es incorrecta ni obsoleta. Puede ser adecuada cuando el tamaño del proyecto, el equipo y los requisitos no justifican una solución distribuida.

Una aplicación monolítica también puede organizarse en tres capas. «Monolítica» describe principalmente la unidad de despliegue, mientras que «tres capas» describe la separación lógica de responsabilidades.

Arquitecturas orientadas a servicios y microservicios

Una **arquitectura orientada a servicios**, o SOA, organiza determinadas capacidades del sistema como servicios accesibles mediante contratos e interfaces definidos. Un servicio puede agrupar una función empresarial amplia y ser utilizado por varias aplicaciones. La organización suele establecer mecanismos comunes de integración, comunicación y seguridad.

Una **arquitectura de microservicios** divide la aplicación en servicios más pequeños, centrados en capacidades concretas y preparados para evolucionar y desplegarse de manera independiente. Cada servicio puede utilizar su propio ciclo de publicación y, cuando resulta apropiado, administrar sus propios datos.

Ambos modelos distribuyen responsabilidades, pero no deben considerarse equivalentes. El tamaño del servicio no constituye la única diferencia. También cambian el grado de autonomía, la organización de los datos, la automatización del despliegue y la forma de supervisar el sistema.

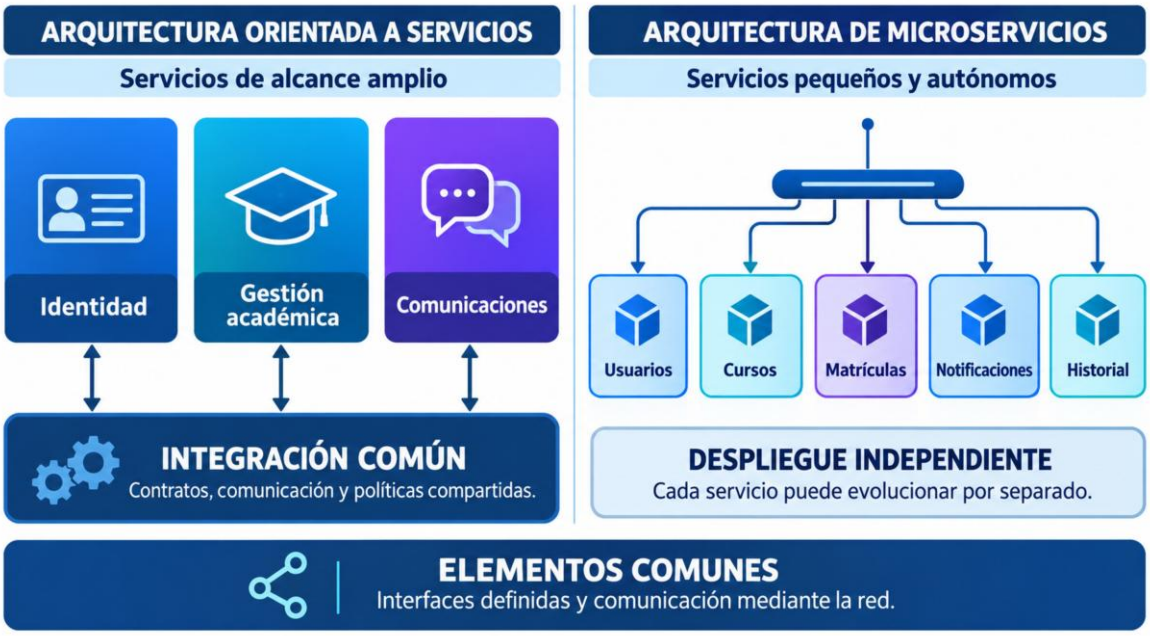
Posibles ventajas:

- Despliegue separado.
- Escalado selectivo.
- Evolución independiente.
- Aislamiento de algunas incidencias.
- Responsabilidad delimitada para cada capacidad del sistema.

Dificultades:

- Comunicación a través de la red.
- Autenticación entre servicios.
- Localización de las instancias que prestan cada servicio.
- Coordinación y coherencia de datos distribuidos.
- Diagnóstico distribuido.
- Mayor esfuerzo de automatización.
- Necesidad de monitorización centralizada.

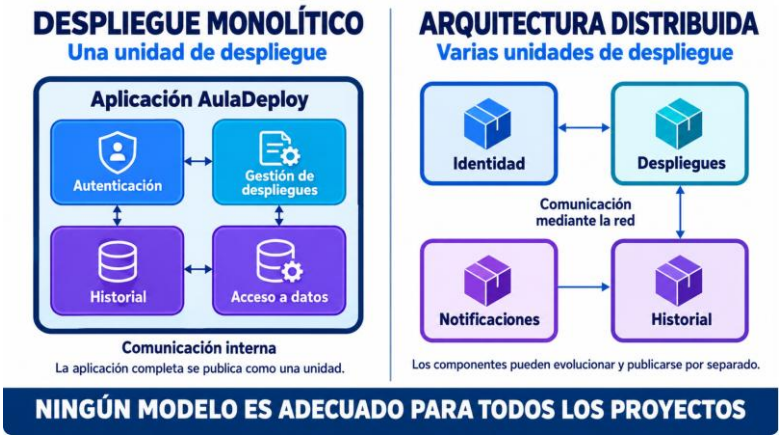
La fragmentación no constituye un objetivo por sí misma. Una aplicación pequeña puede resultar más fiable y fácil de mantener como monolito modular. La división en servicios solo se justifica cuando aporta autonomía, escalado selectivo, aislamiento o capacidad de evolución independiente.



Arquitectura monolítica y arquitectura distribuida

Las arquitecturas monolíticas y distribuidas representan formas diferentes de organizar, desplegar y operar una aplicación. Ninguna de ellas es adecuada en todos los casos. La comparación debe realizarse atendiendo a la unidad de despliegue, la comunicación entre componentes, el escalado, el diagnóstico de errores y la capacidad disponible para administrar la infraestructura. Una solución más distribuida ofrece independencia entre determinadas partes, pero también introduce comunicaciones y tareas de coordinación adicionales.

Criterio	Despliegue monolítico	Arquitectura distribuida
Unidad de despliegue	La aplicación se publica como una unidad principal	Los componentes o servicios pueden publicarse por separado
Comunicación interna	Predomina la comunicación dentro del mismo proceso	Parte de la comunicación se realiza a través de la red
Escalado	Normalmente se replica o amplía la aplicación completa	Determinados componentes pueden escalar de forma independiente
Gestión de versiones	Las partes principales avanzan en una versión conjunta	Debe controlarse la compatibilidad entre versiones diferentes
Diagnóstico	El recorrido de una petición suele concentrarse en una aplicación	Es necesario relacionar registros y fallos de varios componentes
Operación	Requiere menos servicios e infraestructura de coordinación	Necesita más automatización, supervisión y control de comunicaciones
Punto de fallo	Un fallo interno puede afectar al conjunto de la aplicación	Un fallo puede aislarse, pero también propagarse mediante dependencias



La elección debe basarse en los requisitos de la aplicación, el número de usuarios, la disponibilidad necesaria, las funciones que deben escalar, la frecuencia de los cambios y la capacidad del equipo para administrar la infraestructura.

La edición dinámica de AulaDeploy puede organizar presentación, lógica y datos sin convertirse por ello en una aplicación distribuida. Puede mantenerse como un monolito modular desplegado en Tomcat y utilizar una base de datos independiente. Solo tendría sentido dividirla en servicios si aparecieran necesidades reales de escalado, autonomía o integración.

Una arquitectura reciente o compleja no resulta automáticamente más adecuada. La solución debe proporcionar las funciones requeridas con un coste de implantación y mantenimiento asumible.

Componentización y modularidad

A medida que aumenta la funcionalidad de una aplicación, mantener todo su comportamiento como un bloque indiferenciado dificulta su comprensión y modificación. La **componentización** divide el sistema en unidades con responsabilidades definidas e interfaces claras. La **modularidad** persigue que cada una de esas unidades pueda comprenderse, probarse y modificarse sin necesitar un conocimiento completo de los detalles internos del resto de la aplicación.

Un componente puede ser:

- Un módulo de autenticación.
- Un servicio de cursos.
- Una biblioteca.
- Un conector con la base de datos.
- Un conjunto de páginas.
- Un proceso auxiliar.

Desde el punto de vista del despliegue es necesario identificar:

- Qué componentes forman la aplicación.
- Cuáles se despliegan juntos.
- Qué versiones son compatibles.
- Qué configuración necesita cada uno.
- Cómo se comunican.
- Qué ocurre si uno no está disponible.

Para que la división en componentes resulte útil, no basta con crear un gran número de unidades. Cada componente debe agrupar elementos relacionados y mantener dependencias controladas con el resto del sistema.

La **cohesión** expresa hasta qué punto los elementos de un módulo colaboran para cumplir una responsabilidad concreta. Un módulo con buena cohesión agrupa funciones relacionadas. El **acoplamiento** expresa el grado de dependencia entre módulos. Cuando un cambio en un componente obliga a modificar muchos otros, el acoplamiento es elevado y el despliegue resulta más arriesgado.

La componentización no obliga a convertir cada función en un servicio independiente. Una aplicación monolítica puede organizarse mediante módulos bien delimitados. AulaDeploy puede contener módulos de autenticación, gestión de despliegues y consulta del historial dentro del mismo archivo WAR.



Los patrones que el alumnado ya conoce también influyen en el despliegue. Por ejemplo, MVC separa modelo, vista y controlador dentro de la aplicación, pero no obliga a instalar cada parte en un servidor diferente. Un patrón organiza responsabilidades; la arquitectura de despliegue determina dónde se ejecutan.

Escalabilidad vertical y horizontal

El número de usuarios, peticiones y operaciones que debe atender una aplicación puede aumentar con el tiempo. La **escalabilidad** expresa la capacidad del sistema para adaptarse a ese crecimiento manteniendo un funcionamiento aceptable. Esta adaptación puede lograrse asignando más recursos a una misma máquina o aumentando el número de instancias que prestan el servicio. Ambas estrategias resuelven problemas diferentes y presentan límites que deben valorarse antes de modificar la infraestructura.

Escalabilidad vertical

Consiste en asignar más recursos a una misma máquina:

- Más procesador.
- Más memoria.
- Más almacenamiento.
- Mayor capacidad de red.

Su aplicación suele ser directa, pero tiene límites físicos y económicos.

Escalabilidad horizontal

Consiste en aumentar el número de instancias que prestan el servicio. Permite repartir peticiones, pero exige:

- Distribuir el tráfico.
- Controlar el estado compartido.
- Mantener la coherencia.
- Supervisar varias instancias.



Escalar no consiste simplemente en añadir máquinas. Primero debe localizarse el componente que limita el sistema. El problema puede encontrarse en la base de datos, una operación ineficiente o un servicio externo.

Una aplicación preparada para escalar debe evitar depender innecesariamente de información almacenada solo en una instancia.

Rendimiento, disponibilidad y escalabilidad

El funcionamiento de una aplicación no puede evaluarse únicamente comprobando que devuelve una página. También es necesario determinar con qué rapidez responde, durante cuánto tiempo permanece accesible y cómo se comporta cuando aumenta la carga. Estas tres perspectivas se expresan mediante los conceptos de rendimiento, disponibilidad y escalabilidad. Aunque están relacionadas, describen propiedades diferentes y deben analizarse por separado mediante condiciones y medidas comprobables.

Rendimiento

Expresa cómo responde el sistema bajo determinadas condiciones. Puede medirse mediante tiempos de respuesta, operaciones por segundo o consumo de recursos.

Disponibilidad

Indica en qué medida el servicio permanece accesible cuando se necesita.

Escalabilidad

Describe la capacidad del sistema para adaptarse a un aumento de carga.

Estas propiedades no son equivalentes: un sistema puede ser rápido cuando funciona y sufrir interrupciones frecuentes; puede estar disponible pero responder lentamente; puede aceptar más recursos sin aprovecharlos correctamente; puede escalar una capa mientras otra limita el conjunto.

Estas propiedades condicionan:

- El número de instancias.
- La distribución de componentes.
- La necesidad de redundancia.
- La supervisión.
- La recuperación ante fallos.
- La capacidad del almacenamiento.
- La estrategia de actualización.

Los requisitos deben expresarse de forma comprobable. «La aplicación debe ser rápida» no permite evaluar el resultado. Es necesario definir condiciones y valores medibles.

Portabilidad de aplicaciones web

Una aplicación puede tener que trasladarse desde el equipo de desarrollo a un entorno de pruebas, a un servidor de producción o a una infraestructura diferente. La **portabilidad** expresa hasta qué punto ese traslado puede realizarse sin modificar de forma importante el código o la estructura de la aplicación. Una solución portable identifica sus dependencias, separa la configuración del código y evita asumir características que solo existen en un equipo concreto.

La portabilidad mejora cuando:

- Las dependencias están declaradas.
- La configuración está separada del código.
- No se utilizan rutas locales rígidas.
- Se documentan las versiones.
- Se emplean protocolos y formatos estándar.
- El despliegue es reproducible.
- Los datos se gestionan de forma independiente.

No significa que la aplicación funcione automáticamente en cualquier sistema. Pueden existir diferencias de sistema operativo, procesador, permisos, codificación, versiones o servicios.

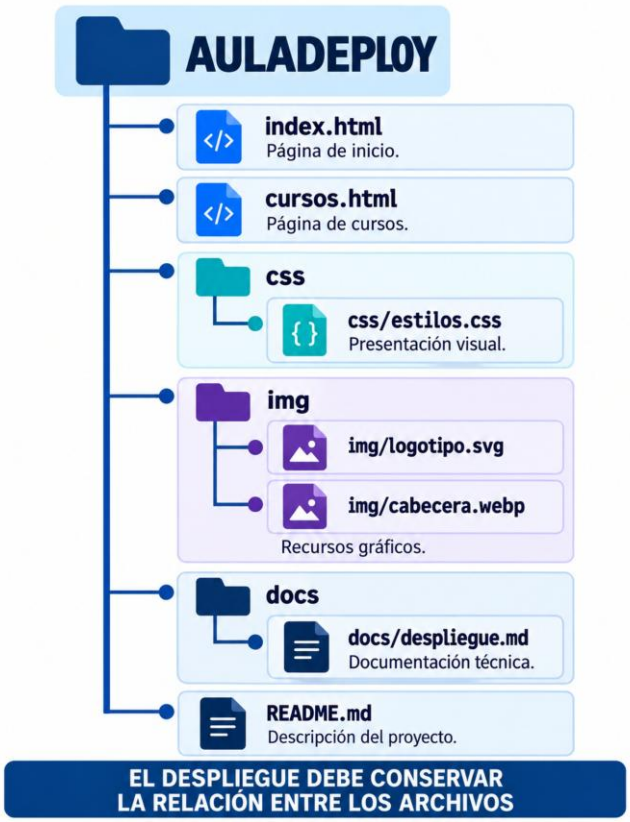
También debe considerarse la dependencia de un proveedor. Utilizar una característica exclusiva puede aportar ventajas, pero dificultar una migración futura. Esta dependencia no es siempre negativa, pero debe conocerse y aceptarse conscientemente.

AulaDeploy presenta una portabilidad elevada porque solo necesita un servidor capaz de entregar HTML, CSS e imágenes. Aun así, puede fallar por diferencias entre mayúsculas y minúsculas, rutas incorrectas o permisos insuficientes.

Estructura de archivos y recursos de una aplicación web

Una aplicación web no suele estar formada por un único documento. Sus archivos mantienen relaciones que deben conservarse durante el despliegue: el HTML referencia hojas de estilo e imágenes, el código puede requerir bibliotecas y la configuración puede señalar recursos externos. Examinar la estructura de directorios permite reconocer qué elementos forman la aplicación, cuáles deben publicarse juntos y qué archivos pertenecen únicamente al proceso de desarrollo.

Estructura inicial de AulaDeploy:



`index.html` es la página inicial. `cursos.html` contiene otra sección. Las carpetas `css` e `img` agrupan los recursos visuales. `README.md` describe el proyecto y `docs` conserva documentación adicional.

El despliegue debe mantener la relación entre los archivos. Copiar `index.html` sin la carpeta `css` permitiría recibir el documento, pero no su presentación visual.

Las aplicaciones dinámicas pueden añadir código ejecutable, plantillas, bibliotecas, migraciones, configuración y artefactos generados.

No todos los archivos del repositorio tienen que publicarse. Las pruebas, fuentes y documentación interna pueden seguir un tratamiento diferente.

Código, recursos, datos y dependencias

Los elementos que forman una aplicación no tienen la misma función ni deben recibir el mismo tratamiento durante el despliegue. El código implementa el comportamiento, los recursos forman parte de la interfaz o del contenido, los datos representan la información gestionada y las dependencias proporcionan capacidades necesarias para la ejecución. Distinguir estas categorías ayuda a decidir qué debe incluirse en el artefacto, qué debe conservarse de manera persistente y qué debe instalarse o configurarse en el entorno.

Código

Contiene las instrucciones que implementan el comportamiento.

Recursos

Incluyen HTML, CSS, imágenes, fuentes, plantillas y documentos.

Datos

Representan la información gestionada por la aplicación. Pueden almacenarse en bases de datos, archivos o servicios externos.

Dependencias

Son bibliotecas, entornos de ejecución y servicios que la aplicación necesita. Deben identificarse por nombre, versión compatible, función, procedencia, condiciones de instalación y configuración necesaria. También pueden existir dependencias transitivas: bibliotecas requeridas por otras bibliotecas.

Confundir código, recursos, datos y dependencias produce despliegues frágiles. Si los datos se guardan en una ubicación temporal, podrían perderse al sustituir la aplicación. Si las dependencias no se declaran, el entorno puede resultar imposible de reproducir.

Configuración y artefactos de despliegue

El código fuente por sí solo no siempre contiene todo lo necesario para poner una aplicación en funcionamiento. El proceso de despliegue debe también proveer los valores específicos de la instalación y un resultado final listo para su publicación. Estos dos componentes esenciales, la **configuración** y el **artefacto de despliegue**, aunque relacionados, cumplen funciones distintas y requieren una gestión independiente para asegurar la flexibilidad y robustez de la aplicación.

Configuración

La configuración abarca todos aquellos valores que pueden variar entre diferentes instalaciones o entornos (desarrollo, pruebas, producción) sin que sea necesario alterar el código fuente de la aplicación. Mantener la configuración separada del código es crucial para la portabilidad y la eficiencia en el ciclo de vida del software.

- Los puertos de red utilizados por la aplicación.
- Las direcciones de otros servicios externos o internos.
- Parámetros para la conexión con la base de datos (credenciales, URL).
- El nivel de registro de eventos (debug, info, warning, error).
- Rutas de almacenamiento de archivos o recursos temporales.
- Límites de recursos asignados (memoria, CPU).
- Referencias a mecanismos seguros para la obtención de credenciales sensibles.

Si un valor como una dirección, una contraseña o un puerto se incrusta directamente en el código, cualquier modificación en el entorno implicaría una nueva edición o reconstrucción de la aplicación, un proceso ineficiente y propenso a errores. Por ello, la configuración debe ser externa y fácil de modificar.

Artefacto de despliegue

Un artefacto de despliegue es el producto final de la construcción de una aplicación, listo para ser publicado y ejecutado en un entorno específico. Este debe ser identificable unívocamente, fácilmente transportable y verificable para asegurar su integridad y consistencia. La forma que adopta un artefacto varía considerablemente según la tecnología y el tipo de aplicación.

- Una carpeta que contiene documentos HTML, hojas de estilo CSS e imágenes.
- Un archivo comprimido (como un .zip o .tar.gz) con todos los recursos de un sitio web estático.
- Un archivo WAR (Web Application Archive) que empaqueta una aplicación web Java.
- Un ejecutable binario acompañado de sus recursos y bibliotecas dependientes.

Es fundamental que el artefacto de despliegue no sea modificado manualmente para adaptarlo a cada instalación. Los valores variables y específicos del entorno deben ser suministrados a través de la configuración externa. De esta manera, se garantiza que el elemento que se publica es exactamente el mismo que ha sido probado y validado, manteniendo la coherencia y fiabilidad del proceso.

Descriptores de despliegue y gestión de secretos

Para que una aplicación web pueda ser cargada, configurada y ejecutada correctamente en una plataforma, a menudo se requieren archivos especiales que describen cómo interactuar con ella. Estos archivos se conocen como **descriptores de despliegue**. Su estructura y contenido son específicos de la tecnología y el entorno de la plataforma, lo que implica que no existe un formato universalmente válido para todas las aplicaciones.

Un descriptor de despliegue suministra a la plataforma información crucial que no está directamente codificada en el ejecutable de la aplicación. Puede especificar componentes, definir parámetros de inicio, configurar rutas, aplicar filtros de seguridad o establecer permisos y otras condiciones esenciales para el funcionamiento. Un ejemplo clásico es el archivo `WEB-INF/web.xml` en aplicaciones basadas en Jakarta Servlet, que declara servlets, filtros y sus correspondencias. Es importante destacar que, aunque parte de esta información puede gestionarse mediante anotaciones en el código, el descriptor sigue siendo un elemento clave en muchos entornos.

Antes de proceder con el despliegue de una aplicación que requiere un descriptor, es fundamental realizar varias verificaciones:

- Confirmar si la plataforma de destino exige el uso de un descriptor de despliegue.
- Identificar qué componente o programa de la plataforma es el responsable de interpretar dicho archivo.
- Verificar la versión y la sintaxis específicas que admite el intérprete.
- Determinar qué información dentro del descriptor debe permanecer estática y cuál puede variar.
- Establecer qué valores específicos deben ser proporcionados mediante mecanismos de configuración externa.

Gestión de Secretos

Un aspecto crítico en la implantación de cualquier aplicación es la **gestión de secretos**. Las contraseñas de bases de datos, las claves privadas de APIs, los tokens de autenticación y cualquier otra información sensible no deben ser parte del código fuente, almacenarse en repositorios públicos, ni incluirse directamente en el artefacto de despliegue. Revelar estos secretos comprometería la seguridad de la aplicación y de los sistemas conectados.

La configuración de la aplicación puede indicar el mecanismo o la ubicación desde donde se obtendrán estos secretos (por ejemplo, un gestor de secretos o variables de entorno), pero nunca debe revelar los valores directamente. Esto asegura que la información confidencial se inyecte en tiempo de ejecución de forma segura y controlada, evitando su exposición en el ciclo de vida del desarrollo y despliegue.

Es importante recordar que el simple hecho de copiar un repositorio completo no constituye un despliegue adecuado. Un repositorio suele contener código fuente, pruebas unitarias, documentación interna y archivos específicos para el entorno de desarrollo que no son necesarios en producción. El proceso de despliegue implica seleccionar o generar un artefacto optimizado que, junto con una configuración externa y la gestión segura de secretos, permita que la aplicación funcione de manera eficiente y protegida en el entorno de destino.



La aplicación final en ejecución es el resultado de la orquestación del código, las instrucciones del descriptor, la adaptación de la configuración externa y la inyección segura de secretos.

Servidores físicos y máquinas virtuales

Los componentes de una aplicación necesitan recursos de procesamiento, memoria, almacenamiento y red para ejecutarse. Estos recursos pueden proporcionarse directamente mediante un servidor físico o mediante máquinas virtuales creadas sobre un sistema de virtualización. Las dos alternativas permiten alojar aplicaciones, pero difieren en la forma de asignar los recursos, aislar los servicios, reproducir los entornos y recuperar el sistema ante una incidencia.

Servidor físico

Es un equipo real cuyos recursos de procesador, memoria, almacenamiento y red se utilizan para ejecutar servicios. El despliegue directo sobre un servidor físico proporciona acceso completo al hardware, pero vincula la aplicación a una máquina concreta. La ampliación, sustitución o recuperación puede requerir intervenciones sobre ese equipo.

Máquina virtual

Es un sistema informático simulado que se ejecuta mediante una capa de virtualización. Dispone de procesador, memoria, discos y red virtuales y ejecuta su propio sistema operativo. El software que crea y administra máquinas virtuales se denomina **hipervisor**.

Ventajas de las máquinas virtuales:

- Aislamiento entre sistemas
- Aprovechamiento compartido del hardware
- Entornos reproducibles
- Facilidad para realizar instantáneas
- Separación de servicios
- Recuperación y migración más flexibles

Limitaciones: cada máquina virtual mantiene su propio sistema operativo; consume memoria y almacenamiento; requiere administrar varias instancias completas.

Una máquina virtual permite aislar AulaDeploy y sus componentes del sistema anfitrión. También facilita conservar una configuración identificable, trasladar el entorno a otro equipo compatible y recuperar un estado anterior mediante una instantánea.



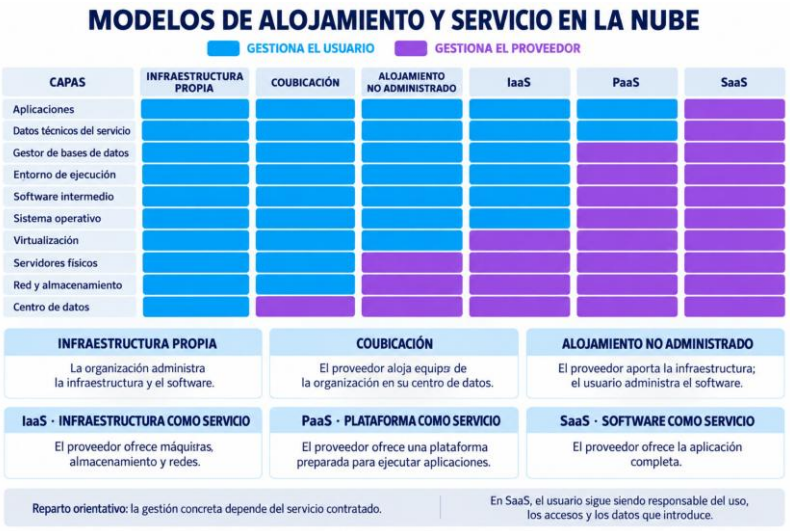
Infraestructuras en la nube

La **computación en la nube** permite acceder mediante una red a recursos informáticos configurables que pueden asignarse y liberarse con rapidez. No designa una máquina concreta. Describe una forma de proporcionar capacidad de procesamiento, almacenamiento, redes, plataformas o aplicaciones como servicios.

Una infraestructura de nube presenta cinco características esenciales:

- **Autoservicio bajo demanda:** el cliente puede solicitar capacidad sin que cada operación requiera una intervención manual del proveedor.
- **Acceso amplio mediante la red:** los servicios se utilizan a través de mecanismos de red normalizados.
- **Agrupación de recursos:** el proveedor comparte su infraestructura entre distintos clientes manteniendo la separación correspondiente.
- **Elasticidad rápida:** los recursos pueden aumentar o disminuir según la demanda.
- **Servicio medido:** el consumo puede supervisarse, controlarse y asociarse a la capacidad utilizada.

Los modelos de servicio indican qué parte del sistema proporciona el proveedor y qué parte continúa bajo la responsabilidad del cliente.



Modelos de servicio más habituales:

IaaS – Infraestructura como Servicio

El proveedor facilita procesamiento, redes, almacenamiento y virtualización. El cliente administra normalmente el sistema operativo, el entorno de ejecución, la aplicación, la configuración y los datos. AulaDeploy podría desplegarse en una máquina virtual contratada como servicio.

PaaS – Plataforma como Servicio

El proveedor administra también el sistema operativo y la plataforma de ejecución. El cliente despliega su aplicación y gestiona su configuración y sus datos. La plataforma limita las versiones, lenguajes y componentes que pueden utilizarse.

SaaS – Software como Servicio

El proveedor ofrece una aplicación completa. El usuario utiliza sus funciones y configura las opciones permitidas, pero no administra la infraestructura ni despliega sobre ella el código de AulaDeploy.

Nube pública, privada, comunitaria e híbrida

Los modelos de despliegue en la nube definen quién es el propietario y gestor de la infraestructura, quién puede acceder a ella y cómo se comparte o integra con otras infraestructuras. Estos modelos son fundamentales para entender la gobernanza, la seguridad y la escalabilidad de los recursos informáticos en la nube.

Nube pública

La infraestructura se ofrece para su utilización por diferentes organizaciones o usuarios. Pertenece normalmente a un proveedor que administra los recursos físicos y los distribuye entre sus clientes. Es importante destacar que "pública" no significa que los datos o aplicaciones de todos los clientes sean visibles entre sí, sino que el acceso a los servicios está disponible para el público en general o una gran comunidad de usuarios a través de internet.

Nube privada

La infraestructura se destina al uso exclusivo de una organización. Puede encontrarse en las propias instalaciones de la organización o estar alojada y administrada por un proveedor externo. Este modelo proporciona a la organización un mayor control sobre la configuración, el acceso, la seguridad y las políticas aplicadas, lo que es crucial para cumplir con regulaciones estrictas o para manejar datos sensibles.

Nube comunitaria

La infraestructura se reserva para varias organizaciones que comparten requisitos, objetivos o condiciones de seguridad. Por ejemplo, distintas instituciones educativas o agencias gubernamentales podrían utilizar una nube preparada para necesidades comunes de protección de datos, interoperabilidad, cumplimiento normativo y administración colaborativa. Los costes y la gestión pueden ser compartidos entre los participantes.

Nube híbrida

Combina dos o más infraestructuras de nube diferenciadas, como una nube privada y una pública. Cada infraestructura mantiene su identidad, pero se conectan para permitir el intercambio o la portabilidad de datos y aplicaciones, por ejemplo, para gestionar picos de demanda (cloud bursting) o para almacenar datos críticos en la nube privada y datos menos sensibles en la pública. No basta con utilizar simultáneamente un servidor local y un servicio de Internet; debe existir una integración planificada entre las infraestructuras.

Es fundamental distinguir entre los modelos de servicio y los modelos de despliegue, ya que responden a preguntas diferentes. El NIST (National Institute of Standards and Technology) contempla precisamente estos cuatro modelos de despliegue como la clasificación estándar.

MODELO DE SERVICIO

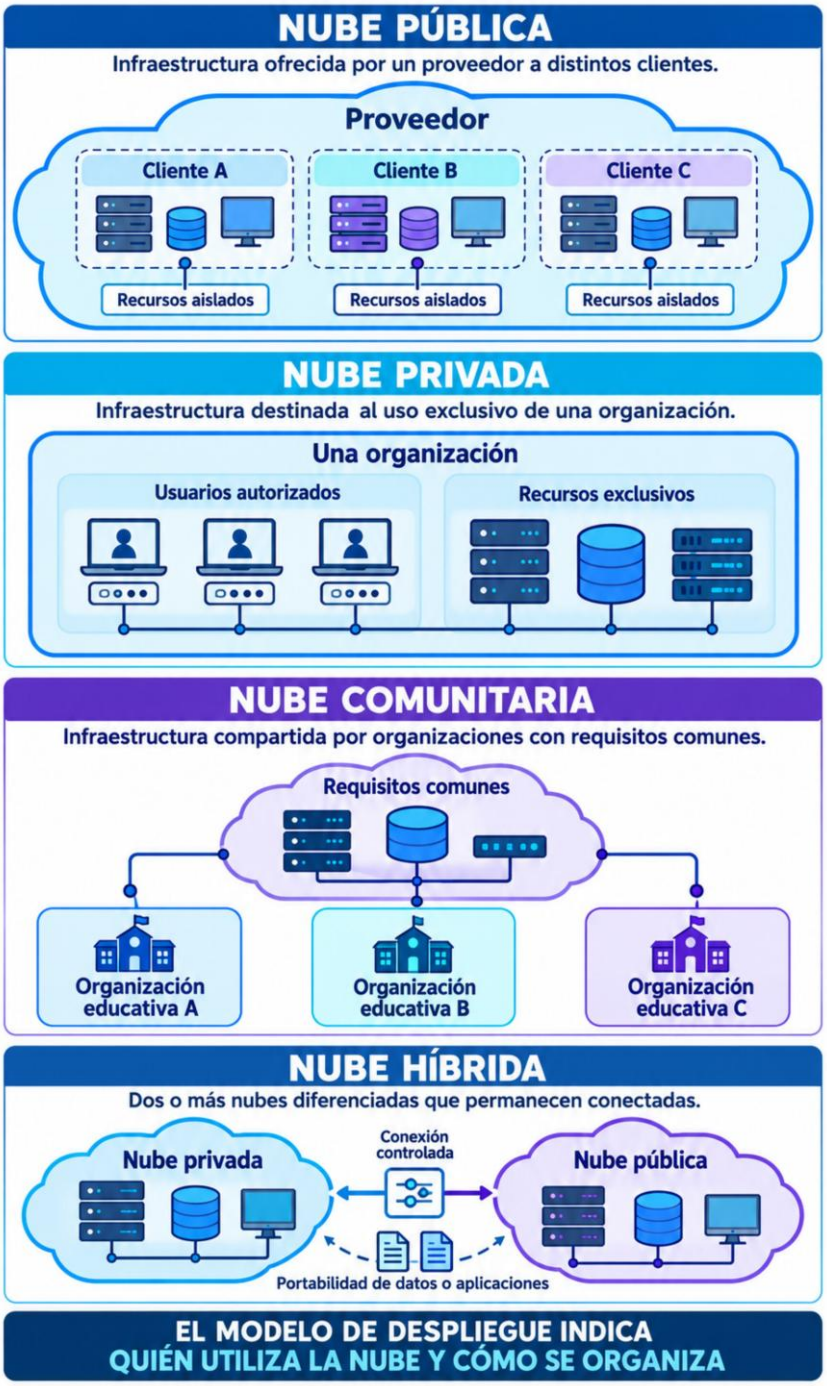
Qué se proporciona (infraestructura, plataforma, software) y quién lo administra (proveedor o cliente).

MODELO DE DESPLIEGUE

Quién utiliza la nube (público, una organización, una comunidad) y cómo se organiza (propia, externa, integrada).

Modelos de despliegue en la nube: esquema comparativo

Los modelos de despliegue en la nube se diferencian por la propiedad de la infraestructura, las personas u organizaciones que pueden utilizarla y el modo en que se combinan sus recursos. La nube pública presta servicios a distintos clientes; la privada se dedica a una organización; la comunitaria se comparte entre entidades con necesidades comunes; y la híbrida integra infraestructuras de más de un modelo. El diagrama resume estas relaciones y debe interpretarse junto con las definiciones de la página anterior.



El modelo de despliegue indica quién utiliza la infraestructura y cómo se organiza; no debe confundirse con el modelo de servicio contratado.

Contenedores

Un **contenedor** es un proceso aislado que se ejecuta con los archivos, bibliotecas y configuración necesarios para una aplicación.

Una **imagen** es el paquete inmutable utilizado como base para crear contenedores. Contiene el sistema de archivos y las instrucciones necesarias para iniciar el proceso.

La imagen y el contenedor no son equivalentes. La imagen es la plantilla inmutable; el contenedor es una instancia en ejecución creada a partir de ella. Si varios contenedores se crean desde una misma imagen, cada uno dispone de su propio estado de ejecución.

Un **volumen** mantiene datos fuera del sistema de archivos temporal del contenedor. En AulaDeploy podría utilizarse para conservar la base de datos o los registros cuando el contenedor se sustituya por otro.

Elementos habituales:

- Imagen.
- Contenedor.
- Motor de contenedores.
- Registro de imágenes.
- Red.
- Volumen.
- Variables de entorno y otros mecanismos de configuración externa.

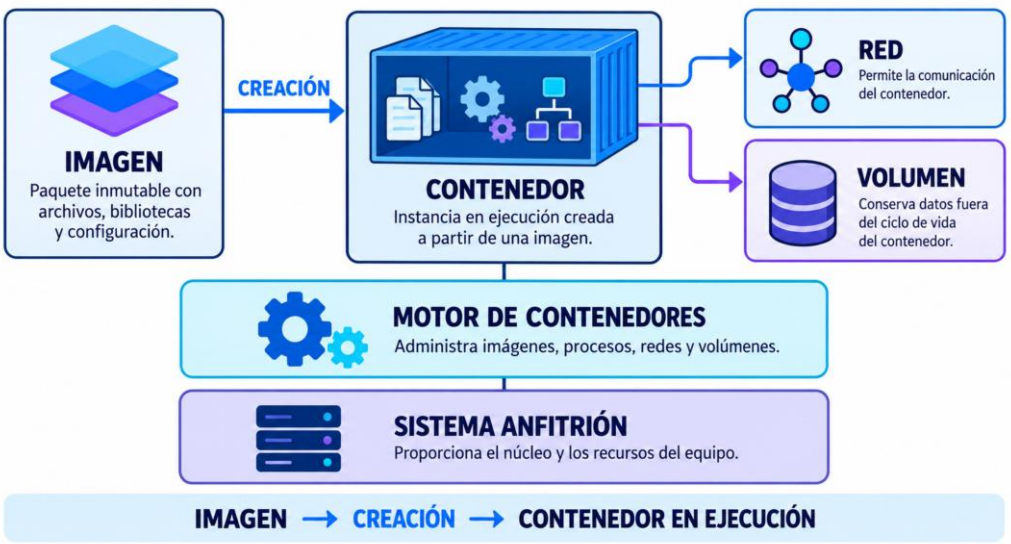
A diferencia de una máquina virtual, un contenedor no ejecuta normalmente un sistema operativo completo independiente. Comparte el núcleo del sistema anfitrión, aunque mantiene aislamiento de procesos, red y almacenamiento.

Ventajas:

- Inicio rápido
- Menor consumo que una máquina virtual completa
- Portabilidad del entorno
- Creación reproducible
- Facilidad para sustituir instancias

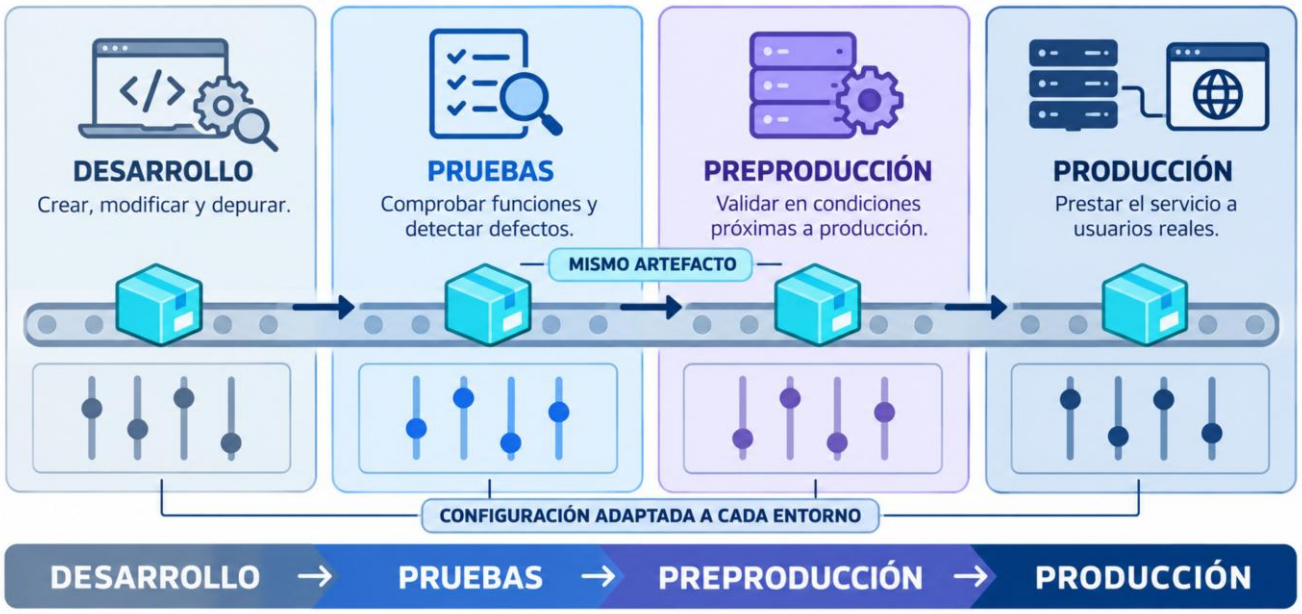
Limitaciones:

- Los datos persistentes requieren volúmenes.
- La seguridad depende de una configuración correcta.
- Debe controlarse la procedencia de las imágenes.
- El aislamiento no equivale necesariamente al de una máquina virtual.



Entornos de desarrollo, pruebas, preproducción y producción

Una aplicación pasa por distintos entornos para separar el trabajo de construcción, comprobación y prestación del servicio. Cada entorno tiene una finalidad, unos usuarios autorizados, una configuración y un conjunto de datos propios.



Desarrollo

Se utiliza para crear, modificar y depurar la aplicación. Puede cambiar con frecuencia y contiene herramientas o configuraciones que no deben trasladarse a producción. Trabaja normalmente con datos ficticios o locales.

Pruebas

Permite ejecutar comprobaciones funcionales, técnicas y de integración sobre una versión identificada. Debe proporcionar resultados repetibles y evitar cualquier conexión accidental con servicios o datos de producción.

Preproducción

Reproduce con la mayor fidelidad razonable las condiciones de producción. Se utiliza para validar el artefacto, la configuración, las comunicaciones, los permisos y el procedimiento de despliegue antes de prestar el servicio real.

Producción

Es el entorno que atiende a los usuarios reales. Requiere controles de acceso, supervisión, copias de seguridad, procedimientos de recuperación y cambios planificados.

Los entornos deben mantenerse separados. No es adecuado utilizar datos reales indiscriminadamente en desarrollo ni conectar pruebas destructivas con servicios de producción.

El mismo artefacto debería avanzar desde pruebas hasta preproducción y producción. Lo que cambia entre entornos es la configuración: direcciones, credenciales, registros, recursos asignados y servicios externos. Si en cada entorno se modifica el código o se construye un artefacto diferente, la versión finalmente publicada no coincide exactamente con la que se ha probado.

Infraestructura física, máquinas virtuales, nube y contenedores

Un servidor físico, una máquina virtual, la computación en la nube y un contenedor no son cuatro alternativas equivalentes. Cada concepto responde a una pregunta distinta: el servidor físico describe el hardware, la máquina virtual representa un sistema completo virtualizado, la nube describe una forma de proporcionar recursos y el contenedor ofrece un entorno aislado para ejecutar procesos. Por ello pueden combinarse dentro de una misma arquitectura, como muestra la tabla siguiente.

Concepto	Qué describe	Qué proporciona	Cómo puede combinarse
Servidor físico	Un equipo real con procesador, memoria, almacenamiento y red	Recursos de hardware sobre los que se instala software	Puede alojar sistemas operativos, máquinas virtuales o contenedores
Máquina virtual	Un sistema informático virtualizado con su propio sistema operativo	Aislamiento y asignación controlada de recursos	Puede ejecutarse en un servidor físico local o proporcionarse mediante la nube
Computación en la nube	Una forma de proporcionar recursos y servicios mediante una red	Aprovisionamiento, elasticidad y consumo medido según el servicio contratado	Puede ofrecer servidores físicos, máquinas virtuales, plataformas o aplicaciones
Contenedor	Un entorno aislado para ejecutar procesos a partir de una imagen	Empaquetado reproducible y separación de procesos, red y almacenamiento	Puede ejecutarse en un servidor físico o dentro de una máquina virtual local o alojada en la nube

Estos conceptos no son excluyentes y no responden a la misma pregunta. Un servidor físico describe el hardware; una máquina virtual describe una forma de virtualizar un sistema completo; un contenedor describe un entorno aislado para ejecutar procesos; y la nube describe cómo se proporcionan y consumen determinados recursos.

AulaDeploy podría ejecutarse en un contenedor situado dentro de una máquina virtual contratada en una nube pública. Para describir correctamente ese escenario deben indicarse todas las capas y no reducirlo a una única etiqueta. La selección depende de los requisitos de capacidad, aislamiento, portabilidad, coste, disponibilidad, administración y recuperación.

No debe confundirse el entorno de producción con la infraestructura utilizada. Producción describe la finalidad del entorno; una máquina virtual, un contenedor o un servicio en la nube describen dónde o cómo se ejecuta.

Requisitos de implantación de una aplicación web

Las arquitecturas, los componentes, las infraestructuras y los entornos estudiados no deben seleccionarse de forma aislada. Cada decisión debe responder a las condiciones que debe cumplir la aplicación y al contexto en el que prestará servicio. Estas condiciones se expresan mediante requisitos de implantación.

Los requisitos relacionan las necesidades del sistema con las decisiones técnicas y proporcionan criterios para comprobar el resultado. Deben formularse de manera concreta y verificable, indicando las funciones necesarias, la compatibilidad, la seguridad, la capacidad y las condiciones de operación. Un requisito impreciso no permite diseñar una prueba ni decidir objetivamente si el despliegue es correcto.

Requisitos funcionales

Operaciones que debe ofrecer la aplicación: consultar cursos, autenticar usuarios o registrar despliegues.

Ejemplo verificable: un usuario con el rol OPERADOR puede registrar un despliegue y consultar posteriormente sus datos y su historial.

Requisitos técnicos

Sistema operativo, servidor, entorno de ejecución, bibliotecas, base de datos, almacenamiento y conectividad.

Ejemplo verificable: el artefacto dinámico de AulaDeploy puede desplegarse en una versión de Tomcat compatible con la versión de Jakarta Servlet utilizada.

Requisitos de seguridad

Autenticación, permisos, protección de credenciales, comunicaciones y tratamiento de los datos.

Ejemplo verificable: un usuario con el rol CONSULTA no puede acceder a las operaciones que modifican o eliminan despliegues.

Requisitos de capacidad

Carga simultánea, volumen de información, transferencia y crecimiento previsto.

Ejemplo de formulación: «La aplicación atenderá a treinta usuarios simultáneos durante una prueba de diez minutos sin devolver errores del servidor». La cifra debe adaptarse al escenario real de la práctica.

Requisitos operativos

Registros, supervisión, copias de seguridad, recuperación, actualización y responsables.

Ejemplo verificable: después del despliegue se comprueba el estado de la aplicación y se conservan registros suficientes para diagnosticar un fallo.

Requisitos de compatibilidad

Navegadores, protocolos, versiones y servicios con los que debe funcionar.

Ejemplo verificable: la interfaz puede utilizarse con los navegadores definidos para el aula y todas sus operaciones esenciales funcionan sin JavaScript.

Un requisito debe expresar una condición que pueda comprobarse. «El servidor debe ser rápido» no permite diseñar una prueba. En cambio, una condición que indique número de usuarios, duración, tiempo de respuesta o porcentaje de errores permite obtener una evidencia y decidir si el requisito se cumple.



Referencias y recursos de consulta

RFC 3986: Uniform Resource Identifier (URI): Generic Syntax

Internet Engineering Task Force. Especificación de la sintaxis general de las URI y de la relación entre URI, URL y URN.
https://www.rfc-editor.org/rfc/rfc3986

RFC 9110: HTTP Semantics

Internet Engineering Task Force. Especificación de la semántica de HTTP, los métodos, las cabeceras y los códigos de estado.
https://www.rfc-editor.org/rfc/rfc9110

Generalidades del protocolo HTTP — MDN Web Docs

Introducción al intercambio de peticiones y respuestas HTTP.
https://developer.mozilla.org/es/docs/Web/HTTP/Guides/Overview

¿Qué es un servidor web? — MDN Web Docs

Descripción del funcionamiento básico de los servidores web.
https://developer.mozilla.org/es/docs/Learn_web_development/Howto/Web_mechanics/What_is_a_web_server

Apache HTTP Server Documentation

Apache Software Foundation. Documentación oficial del servidor web Apache HTTP Server.
https://httpd.apache.org/docs/

Apache Tomcat Documentation

Apache Software Foundation. Documentación oficial del contenedor web Apache Tomcat.
https://tomcat.apache.org/

The NIST Definition of Cloud Computing. Special Publication 800-145

National Institute of Standards and Technology. Definición de la computación en la nube y de sus modelos principales.
https://doi.org/10.6028/NIST.SP.800-145

What is a container? — Docker

Introducción al concepto de contenedor y a sus diferencias respecto a una máquina virtual.
https://docs.docker.com/get-started/docker-concepts/the-basics/what-is-a-container/